

■ 2.2.9 Making Replacements for Patterns

You can think of a mathematical relation as being a statement that two mathematical expressions are equal. So, for example, the trigonometric formula $\sin(2x) = 2 \sin(x) \cos(x)$ states that the sine of a double angle is equal to a product of trigonometric functions.

From a formula like this, you can derive several different transformation rules for trigonometric functions. Two of the possible rules are:

`Sin[2 x_] -> 2 Sin[x] Cos[x]` and `Sin[x_] Cos[x_] -> Sin[2 x] / 2.`

These rules effectively use the trigonometric formula “in opposite directions”. Depending on exactly what calculation you are doing, you may want to use either one of these rules.

Making a *Mathematica* definition such as `Sin[2 x_] := 2 Sin[x] Cos[x]`, specifies that *whenever* an expression of the form `Sin[2 x_]` is encountered, it should be replaced by `2 Sin[x] Cos[x]`. Often, however, you do not want a transformation like this to be made in such an automatic way. Instead, you may simply want to set up the rule, and then apply it only “on request”.

You can use *replacements* to specify transformation rules that are to be made in this kind of controlled way. Section 1.3.3 discussed making replacements for symbols. Like definitions, replacements in *Mathematica* can be made for any kinds of expression, including a pattern.

Here is an expression involving trigonometric functions.

```
In[1]:= t = Sin[2 x] + Sin[2 y z] + Sin[z]
Out[1]= Sin[2 x] + Sin[z] + Sin[2 y z]
```

This makes a replacement for expressions that match the pattern `Sin[2 x_]`.

```
In[2]:= t /. Sin[2 x_] -> 2 Sin[x] Cos[x]
Out[2]= 2 Cos[x] Sin[x] + Sin[z] + 2 Cos[y z] Sin[y z]
```

This makes a replacement for all expressions of the form `Sin[x_]`.

```
In[3]:= t /. Sin[x_] -> 2 Sin[x/2] Cos[x/2]
Out[3]= 2 Cos[x] Sin[x] + 2 Cos[ $\frac{z}{2}$ ] Sin[ $\frac{z}{2}$ ] +
      2 Cos[y z] Sin[y z]
```

You will often find it convenient to give “names” to transformation rules that you are going to use in replacements.

This assigns the “name” `sinexp` to the transformation rule.

```
In[4]:= sinexp = Sin[2 x_] -> 2 Sin[x] Cos[x]
Out[4]= Sin[2 (x_)] -> 2 Cos[x] Sin[x]
```

The value of `sinexp` is the transformation rule.

```
In[5]:= sinexp
Out[5]= Sin[2 (x_)] -> 2 Cos[x] Sin[x]
```

Now the rule can be requested “by name”.

```
In[6]:= Sin[2 (1 + x)^2] /. sinexp
Out[6]= 2 Cos[(1 + x)^2] Sin[(1 + x)^2]
```

<code>pattern := value</code>	define a transformation rule to be used automatically
<code>rule = pattern -> value</code>	give a name to a transformation rule
<code>expression /. rule</code>	apply a named rule

Functions for setting up replacements.

You can also use replacements for patterns to do some simple kinds of “mathematical editing” on expressions. For example, you can drop all terms of the form $x^{_}$ in a sum, just by applying a rule that replaces them with 0.

This expression involves various powers of x .

```
In[7]:= Expand[(1 + x + y)^3]
Out[7]= 1 + 3 x + 3 x^2 + x^3 + 3 y + 6 x y + 3 x^2 y +
        3 y^2 + 3 x y^2 + y^3
```

Making this replacement drops all terms involving powers of x in the expression.

```
In[8]:= % /. x^_ -> 0
Out[8]= 1 + 3 y + 6 x y + 3 y^2 + 3 x y^2 + y^3
```

You should realize that the rules discussed here make transformations based on the *structure* of expressions. Section 3.4.10 discusses the function `AlgebraicRules`, which allows you to set up rules based directly on *algebraic form* of expressions.